

Heya AI – Voice Analytics Platform

A multi-tenant SaaS platform that processes AI voice agent call recordings and surfaces call intelligence dashboards to clients. Built as part of RMIT COSC2667/2777, Semester 4.

****Handover document – Heya AI internal team, May 2026****

Table of Contents

- [Overview](#overview)
- [Features](#features)
- [Architecture](#architecture)
- [Tech Stack](#tech-stack)
- [Prerequisites](#prerequisites)
- [Project Structure](#project-structure)
- [Environment Setup](#environment-setup)
- [Database Setup](#database-setup)
- [Data Ingestion](#data-ingestion)
- [Audio Pipeline](#audio-pipeline)
- [RAG Setup](#rag-setup)
- [Agent Analysis (ReAct)](#agent-analysis-react)
- [Running the Application](#running-the-application)
- [Demo Accounts](#demo-accounts)
- [API Reference](#api-reference)
- [Frontend Pages](#frontend-pages)
- [Security Model](#security-model)
- [Quality Scoring](#quality-scoring)
- [Alert System](#alert-system)
- [Export & Reports](#export--reports)
- [Known Gotchas](#known-gotchas)
- [Test Suite](#test-suite)

Overview

Heya AI is a multi-tenant voice analytics platform for businesses that use AI voice agents for phone calls. It ingests call recordings from platforms like Retell AI, runs an audio intelligence pipeline to extract acoustic features, and presents the results through an analytics dashboard.

Three core capabilities:

1. ****Audio Intelligence Pipeline**** – pyannote speaker diarisation + librosa acoustic features extract engagement scores, sentiment trajectory, silence ratios, interruption counts, and emotion detection from raw `.wav` files.
2. ****RAG Conversational Interface**** – pgvector embeddings + Cerebras LLM (Qwen3 235B) let users ask natural language questions over their call data ("what were the top complaints last week?", "which calls had frustrated customers?").
3. ****ReAct Agent Analysis**** – a multi-step reasoning agent with 4 specialised tools that performs deep per-agent and per-client analysis, with a transparent reasoning chain shown to the user.

The platform serves two demo clients:

- ****Artel Apartments**** – Melbourne property management (314 calls, fully processed)
- ****MVAA Legal**** – Melbourne motor vehicle accident law firm (~421 calls)

Features

Client Dashboard

- KPI summary cards: engagement score, success rate, silence ratio, call volume, avg duration
- Engagement benchmark comparison against Voice AI industry baseline (62.0)
- 5 embedded charts: engagement distribution, sentiment trajectory, calls over time, call flow breakdown, outcome vs trajectory
- Call-level alert detection: early disconnects, poor engagement, abusive language, extreme silence, high interruptions
- Filterable call table with agent column, full-text search, and call detail drawer with audio playback
- Live call feed that auto-polls every 5 seconds
- Full-text transcript search with highlighted snippet previews
- Natural language "Ask Your Data" with multi-turn RAG chat and per-agent ReAct analysis
- Light / dark theme toggle – persists to localStorage

Audio Insights (9 Sections)

- Emotion detection from vocal pitch/energy patterns (not transcript text)
- Engagement scoring (0–100 composite acoustic score)
- Agent/customer talk time distribution
- Conversation flow classification (smooth / moderate / poor)
- Silence ratio analysis with success correlation callout
- Sentiment trajectory (improving / stable / deteriorating)
- Speaking rate analysis (words per minute)
- Acoustic stress detection with coaching-focused framing
- Quality grade distribution (A–F)

Admin (Heya Admin) View

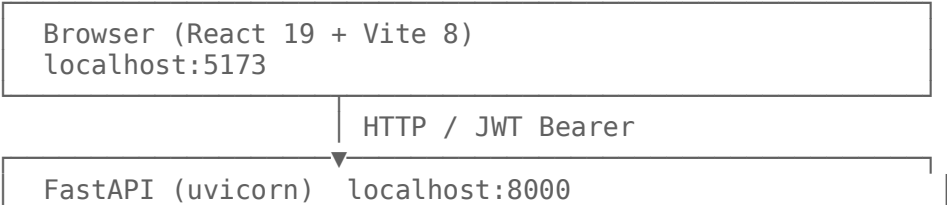
- God View: all clients in one overview matrix with KPIs
- Per-client drill-down panel with 7 analytics metrics bar
- Cross-client call feed with 18-column table, 9 filter params, CSV export, live 5s auto-refresh
- Admin RAG: "Platform Overview (All Clients)" mode – cross-client analysis and comparison
- Dynamic cross-client insight callout (e.g., silence ratio comparison across clients)
- User management: create/delete users, assign to clients
- Alert configuration per client

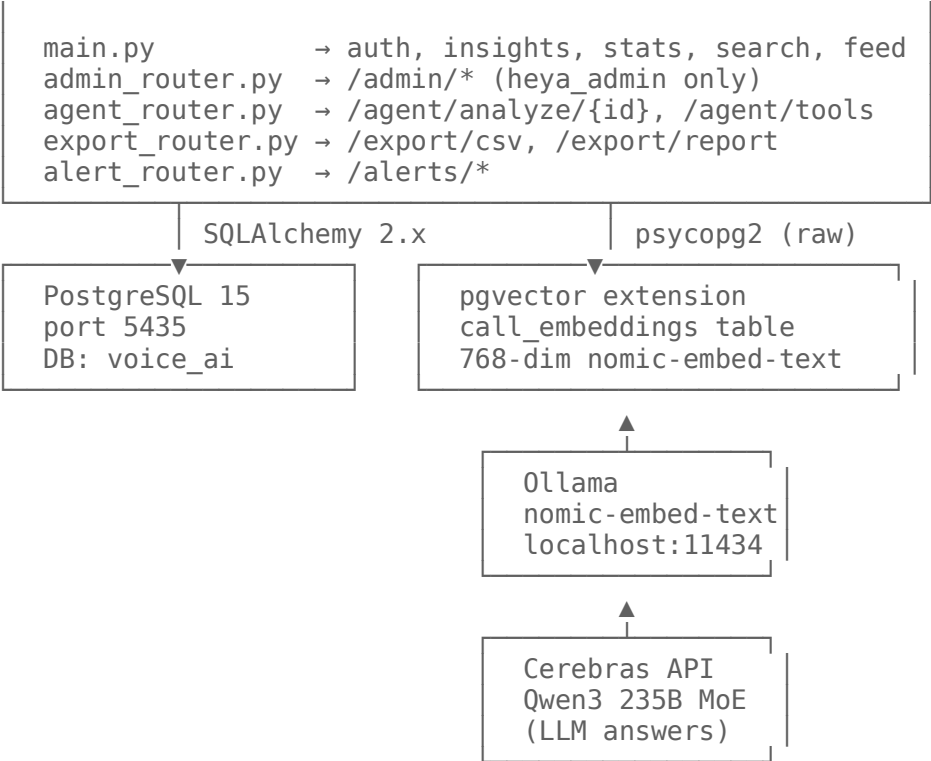
Export

- CSV download of all call data + audio insights (UTF-8 BOM for Excel)
- Printable HTML report with KPI cards, engagement benchmark label, and summary stats

Architecture

^^^





Offline pipeline (heya_pipeline conda env – GPU):

```
pipeline.py      pyannote diarisation
librosa          acoustic features
emotion_processor.py  emotion2vec
→ writes to audio_insights table
```

...

Tech Stack

Layer	Technology	Version
Backend	FastAPI + uvicorn	FastAPI 0.135, uvicorn 0.42
Language	Python	3.11
Database	PostgreSQL	15, port **5435**
ORM	SQLAlchemy	2.x
Vector search	pgvector	768-dim embeddings
Audio pipeline	pyannote.audio	3.x
Acoustic features	librosa	latest
Emotion detection	emotion2vec	via transformers
ML framework	PyTorch	CUDA-enabled
Embeddings	Ollama (nomic-embed-text)	local
LLM	Cerebras API – Qwen3 235B	via openai SDK
Auth	python-jose JWT HS256 + bcrypt	direct (no passlib)
Frontend	React	19
Build tool	Vite	8
Routing	react-router-dom	v7
Charts	recharts	3.x
Animation	framer-motion	12.x
HTTP client	axios	1.x

Prerequisites

Required Software

Tool	Version	Notes
Python	3.11	via Miniconda/Anaconda
Node.js	18+	for frontend
PostgreSQL	15	on port **5435** (not 5432)
Ollama	latest	for RAG embeddings
CUDA toolkit	11.8+	optional, for faster pipeline

Required API Keys

Service	Purpose	Where
Cerebras API	LLM for RAG answers and agent analysis	cloud.cerebras.ai
Hugging Face	pyannote model download	huggingface.co – must accept pyannote terms

Conda Environments

Two separate environments are required. They ****cannot be merged**** due to Windows GPU DLL conflicts:

Environment	Purpose
`heya_audio`	Web server, FastAPI, RAG, embeddings
`heya_pipeline`	Audio pipeline – pyannote, librosa, torch (GPU)

Project Structure

project/	
backend/	
main.py	# FastAPI app – all HTTP endpoints + folder watcher
database.py	# SQLAlchemy models, session factory, CRUD helpers
auth.py	# JWT encode/decode, bcrypt, CurrentUser dataclass
auth_router.py	# POST /auth/login
admin_router.py	# /admin/* (heya_admin only) – overview, feed, calls, agents,
users	
agent_router.py	# /agent/analyze/{id}, /agent/tools (ReAct engine)
alert_router.py	# /alerts/*
export_router.py	# GET /export/csv/{id}, /export/report/{id}
rag.py	# RAG handler – SQL stats + pgvector + Cerebras LLM
agent.py	# ReAct engine – 4 tools, client-scoped system prompt
recommendations.py	# generate_recommendations() + generate_client_alerts()
quality.py	# compute_quality_score() + quality_grade()
pipeline.py	# pyannote → acoustic features (heya_pipeline env)
emotion_processor.py	# emotion2vec inference (heya_pipeline env)
topic_classifier.py	# keyword-based call topic classification
webhook_processor.py	# Retell webhook → DB ingestion
ingest_dataset.py	# bulk ingest from dataset/ folder
run_pipeline.py	# batch audio pipeline runner
run_single_pipeline.py	# single-call pipeline (used by watcher + upload)
rebuild_rag.py	# rebuild pgvector embeddings for all clients
seed_users.py	# create demo users

```

├── migrate_db.py          # core schema migrations
├── migrate_pgvector.py    # call_embeddings table
├── migrate_rls.py         # PostgreSQL Row Level Security
├── migrate_emotion.py     # emotion columns on audio_insights
├── migrate_search.py      # full-text GIN indexes
├── migrate_topics.py      # topic column on calls
├── .env                  # secrets (gitignored)
├── frontend/
│   └── src/
│       ├── main.jsx      # entry – all routes, ProtectedRoute, RootRedirect
│       ├── App.css       # SHARED stylesheet (imported by all pages)
│       ├── index.css     # CSS variables for dark/light themes (data-theme on <html>)
│       ├── api/apiClient.js # axios with auto Bearer token
│       ├── context/
│       │   ├── AuthContext.jsx # JWT in localStorage, login(), logout(), ready flag
│       │   ├── FilterContext.jsx # global filter state (date, flow, direction, topic, agent)
│       │   └── ThemeContext.jsx # dark/light theme toggle, persists to localStorage
│       ├── hooks/useClientData.js # fetches /stats + /insights for all client pages
│       ├── components/
│       │   ├── Sidebar.jsx    # nav sidebar with theme toggle button at bottom
│       │   ├── MotionDrawer.jsx # sliding call detail panel (framer-motion)
│       │   └── AudioPlayer.jsx # in-drawer audio playback
│       ├── pages/
│       │   ├── Login.jsx
│       │   └── AdminApp.jsx    # heya_admin god view – 4 tabs
│       └── client/
│           ├── ClientLayout.jsx
│           ├── Home.jsx      # KPI cards + 5 charts + alerts + calls
│           ├── Calls.jsx     # filter bar (incl. agent) + table + drawer
│           ├── AudioInsights.jsx # 9 analytics sections
│           ├── Trends.jsx    # time-series charts
│           ├── AskYourData.jsx # RAG chat + Agent Analysis section
│           ├── Feed.jsx      # live call feed
│           ├── Alerts.jsx    # alert log
│           └── Search.jsx     # full-text search
├── dataset/
│   ├── artel_apartments/    # Artel call data + recordings
│   └── mvaa_legal/          # MVAA call data + recordings
├── tests/                  # pytest suite – 15 test files
├── docs/                  # full technical documentation + SOW + proposal
├── CLAUDE.md              # AI assistant instructions for this codebase
├── requirements.txt        # Python deps (heya_audio env)
├── README.md              # this file
└── ...

```

Environment Setup

1. Create the heya_audio Conda Environment

Runs the web server, RAG, and all API endpoints.

```

```bash
conda create -n heya_audio python=3.11
conda activate heya_audio
pip install fastapi uvicorn sqlalchemy psycpg2-binary python-jose bcrypt python-dotenv

```

```
pydantic
pip install langchain-core langchain-community openai requests pgvector
\\`
```

### ### 2. Create the heya\_pipeline Conda Environment

Runs the GPU audio pipeline. Do NOT install these packages into `hey\_audio`.

```
\\`bash
conda create -n heya_pipeline python=3.11
conda activate heya_pipeline
conda install pytorch torchvision torchaudio pytorch-cuda=11.8 -c pytorch -c nvidia
pip install pyannote.audio librosa transformers funasr
\\`
```

### ### 3. Install Ollama

Required for local embedding generation (RAG).

```
\\`powershell
Start Ollama server
ollama serve

Pull the embedding model (one-time)
ollama pull nomic-embed-text
\\`
```

Ollama must be running at `http://localhost:11434` before using any RAG endpoints. All other dashboard features work without it.

### ### 4. Install Frontend Dependencies

```
\\`bash
cd frontend
npm install
\\`
```

---

## ## Environment Variables

Create `backend/.env`. This file is gitignored and must never be committed.

```
\\`env
PostgreSQL – note port 5435, not 5432
DATABASE_URL=postgresql://postgres:<password>@localhost:5435/voice_ai

App-role connection (Row Level Security enforced)
APP_DATABASE_URL=postgresql://hey_a_app:<password>@localhost:5435/voice_ai

Cerebras LLM (for RAG answers and agent analysis)
CEREBRAS_API_KEY=<your-cerebras-api-key>

Optional: switch model if 235B is queued
CEREBRAS_MODEL=llama3.1-8b

Hugging Face token (pyannote diarisation model download)
HF_TOKEN=<your-huggingface-token>

JWT signing secret – change in production
```

```
JWT_SECRET_KEY=heya-ai-voice-analytics-secret-2026-rmit
```

```
Path to GPU conda env Python (for subprocess calls from web server)
PIPELINE_PYTHON=C:\Users\Bhanu\miniconda3\envs\heya_pipeline\python.exe
```
```

```
---
```

Database Setup

1. Create the Database

Connect to PostgreSQL as superuser and run:

```
```sql
CREATE DATABASE voice_ai;
\c voice_ai
CREATE EXTENSION IF NOT EXISTS vector;
```
```

2. Run Schema Migrations

Run from `backend/` with `heya\_audio` activated, in this order:

```
```bash
conda activate heya_audio
cd backend

python migrate_db.py # core schema: clients, agents, calls, audio_insights, etc.
python migrate_pgvector.py # call_embeddings table for RAG
python migrate_emotion.py # emotion columns on audio_insights
python migrate_search.py # full-text search GIN indexes on transcripts
python migrate_topics.py # topic column on calls
python migrate_rls.py # Row Level Security (recommended for production)
```
```

3. Seed Demo Users

```
```bash
python seed_users.py
```
```

Key Schema Notes

```
```sql
clients - id (PK), name, folder_name, created_at
agents - id (PK), client_id (FK), name, version
calls - id (PK), client_id, agent_id, direction,
 start_timestamp, ← one 't' – DB typo, never rename
 duration_ms, processing_status, call_successful, topic,
 user_sentiment ← 'Positive' | 'Negative' | 'Neutral' (capitalised)
audio_insights - call_id (PK/FK), engagement_score, silence_ratio,
 interruption_count, hesitation_count, conversation_flow,
 sentiment_trajectory, quality_score, quality_grade,
 user_sentiment, dominant_emotion, agent_talk_time_sec,
 customer_talk_time_sec, total_silence_sec, avg_pitch_hz,
 agent_avg_pitch_hz, customer_avg_pitch_hz, avg_energy
transcript_utterances - id, call_id (FK), role (agent|user),
 content, emotion, emotion_score
call_embeddings - call_id, client_id, embedding (vector(768)), summary
```
```

```
rag_query_history – client_id, user_id, query, response, query_type, response_time_ms
```

> ****Important:**** `start\_timestamp` has a one-letter typo from the original Retell export. It is aliased to `start\_timestamp` in Python. ****Never rename this column.****

> ****Important:**** `user\_sentiment` values are capitalised in the database (`'Positive'`, `'Negative'`, `'Neutral'`). Always `.toLowerCase()` before comparing in frontend CSS or JavaScript objects.

Data Ingestion

`ingest\_dataset.py` loads call metadata and transcripts from the `dataset/` folder into PostgreSQL.

Phase 1 – Metadata & Transcripts (Fast, no GPU)

```
```bash
conda activate heya_audio
cd backend

python ingest_dataset.py --phase 1 # ingest all calls
python ingest_dataset.py --phase 1 --limit 5 # test on first 5 calls
python ingest_dataset.py --summary # check row counts
```
```

Phase 2 – Audio Pipeline (Slow, GPU)

```
```bash
conda activate heya_pipeline
python ingest_dataset.py --phase 2
```
```

Webhook Ingestion (Live / Real-time)

Point Retell AI webhook at `POST /webhook/retell`. Calls are auto-ingested immediately when the call ends. HMAC-SHA256 verified.

Audio Pipeline

Runs in the `heya\_pipeline` conda environment using PyTorch/CUDA.

What It Computes

| Feature | Method |
|----------------------------|---|
| Speaker segments | pyannote.audio 3.x diarisation |
| Engagement score (0–100) | Composite: pitch variation, energy, speech rate |
| Silence ratio | Total silence / call duration |
| Interruption count | Speaker overlaps from diarisation |
| Hesitation count | Short within-speech pauses |
| Agent / customer talk time | Sum of speaker-labelled segments |
| Average pitch (Hz) | librosa `yin` estimator per utterance |
| Average energy | RMS energy per frame |
| Conversation flow | smooth / moderate / poor |
| Sentiment trajectory | improving / stable / deteriorating |

| Dominant emotion | emotion2vec model – acoustic, not text-based |

Running the Pipeline

```
```bash
conda activate heya_pipeline
cd backend

Process all calls with processing_status = 'pending_audio'
python run_pipeline.py

Run emotion post-processor (fills missing dominant_emotion)
python emotion_processor.py --client client_hey_a_001
python emotion_processor.py --client client_hey_a_002

Use --limit / --offset for chunked runs if needed
python emotion_processor.py --client client_hey_a_001 --limit 50 --offset 0
```
```

> On Windows, always set `setenv PYTHONIOENCODING = "utf-8"` before running pipeline scripts – emoji in the codebase causes `UnicodeEncodeError` with cp1252 encoding.

pyannote Authentication

1. Create a Hugging Face account and accept the terms for `pyannote/speaker-diarization-3.0`
2. Generate an access token and add `HF\_TOKEN=<token>` to `backend/.env`

RAG Setup

The RAG system combines pgvector semantic search with direct SQL statistics to answer natural language questions with zero hallucination.

How It Works

| Layer | What it does |
|-------------------|--|
| --- --- | |
| **SQL Fast-Path** | Direct SQL for exact lookups – no LLM |
| **VERIFIED** | SQL aggregates injected as context; LLM formats, never invents numbers |
| **SEMANTIC** | pgvector cosine similarity search retrieves relevant call summaries |

Admin Cross-Client Mode

When `client\_id = "hey\_a\_admin"` is sent in the RAG request, the system switches to platform-wide mode: it aggregates stats across all clients, builds a cross-client context block, and allows comparison questions ("which client has better engagement?"). The "Platform Overview (All Clients)" option in the Admin Intelligence tab triggers this.

Building the Vector Store

Ollama must be running before building embeddings.

```
```bash
Option 1: via API
POST /embed/client_hey_a_001
Authorization: Bearer <admin-token>

Option 2: direct script
conda activate heya_audio
```

```
cd backend
python rebuild_rag.py
\`\`\`
```

### ### Cerebras LLM

```
- **Primary model:** `qwen-3-235b-a22b-instruct-2507`
- **Fallback:** `llama3.1-8b` – set `CEREBRAS_MODEL=llama3.1-8b` in `.env` if queued
- **Rate limit:** 60,000 tokens/minute
- **SDK:** `openai` Python package with `base_url="https://api.cerebras.ai/v1"`

> **Warning:** Do NOT use LangChain LLM wrappers in `heya_audio`. A bug in `langchain-core`
0.3.86` causes a crash on LLM instantiation. Use `_call_llm(prompt)` in `rag.py` which calls
Cerebras directly via the openai SDK.
```

```

```

## ## Agent Analysis (ReAct)

A multi-step reasoning agent built with LangChain ReAct that analyses agent performance with full tool-call transparency.

### ### How It Works

The agent (`agent.py`) has 4 specialised tools:

Tool	Description
`get_call_stats`	Aggregate statistics for a specific agent – volume, success rate, avg engagement
`get_engagement_trend`	Engagement score trend over time
`get_sentiment_breakdown`	Sentiment distribution (Positive / Negative / Neutral)
`get_sample_calls`	Retrieve sample calls filtered by outcome or quality

The system prompt is **client-scoped** – the agent only knows about the persona names that belong to the requesting client. A MVAA user cannot see Artel's agent names, and vice versa.

### ### Frontend Integration

Agent Analysis is integrated into the **Ask Your Data** page (`/dashboard/ask`) as a second section below the RAG chat:

- Client-specific agent cards are shown (Artel sees only Sasha; MVAA sees Justine, Sarah, Julia)
- Preset analysis pills trigger canned prompts per agent
- Agent answers in chat show: confidence badge, tools\_used chips, step count, latency, collapsible reasoning chain

### ### Endpoints

```
\`\`\`
```

```
POST /agent/analyze/{client_id}
Body: { "query": "How is Sasha performing?", "agent_name": "Sasha" }
Returns: { "answer": "...", "tools_used": [...], "steps": [...], "confidence": "HIGH" }
```

```
GET /agent/tools
Returns: list of available tool names and descriptions
\`\`\`
```

```

```

## ## Running the Application

### ### Terminal 1 – Backend

```

```powershell
$env:PYTHONIOENCODING = "utf-8"
conda activate heya_audio
cd D:\rmit\semester_4\project\backend
python -m uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

```

On startup: creates tables if missing, checks Ollama health, starts folder watcher daemon.

> Sign out and sign in once after a backend restart to refresh the JWT with the latest `name` field.

### ### Terminal 2 – Frontend

```

```powershell
cd D:\rmit\semester_4\project\frontend
npm run dev
# Opens at http://localhost:5173
```

```

### ### Terminal 3 – Ollama (RAG only)

```

```powershell
ollama serve
```

```

Required only for "Ask Your Data" queries and embedding rebuilds.

---

## ## Demo Accounts

| Email                  | Password          | Role       | Access                 |
|------------------------|-------------------|------------|------------------------|
| `admin@heya.au`        | `heya_admin_2026` | heya_admin | All clients – God View |
| `admin@artel.com`      | `artel_2026`      | client     | Artel Apartments only  |
| `admin@mvaallegal.com` | `mvaa_2026`       | client     | MVAA Legal only        |

### ### Demo Clients

| Client ID         | Name             | Agents                | Calls | Status          |
|-------------------|------------------|-----------------------|-------|-----------------|
| `client_heya_001` | Artel Apartments | Sasha (inbound)       | 314   | Fully processed |
| `client_heya_002` | MVAA Legal       | Justine, Sarah, Julia | ~421  | Fully processed |

### #### Artel Apartments

Property management company in Brunswick, Melbourne. Sasha handles all inbound calls 24/7 – check-in instructions, booking inquiries, property FAQs, guest identity verification, secure SMS forms, human escalation.

### #### MVAA Legal

Motor vehicle accident law firm in Melbourne.

| Agent   | Direction | Role                                                  |
|---------|-----------|-------------------------------------------------------|
| Justine | Inbound   | Front desk – inquiries, case updates, routing         |
| Sarah   | Outbound  | Calls at-fault drivers to collect insurance details   |
| Sarah   | Inbound   | Handles return calls from at-fault drivers            |
| Julia   | Outbound  | Checks whether clients have signed their Welcome Pack |
| Julia   | Inbound   | Handles return calls from Welcome Pack follow-up      |

---

## ## API Reference

All endpoints except `POST /auth/login` require `Authorization: Bearer <jwt-token>`.

### ### Authentication

| Method | Path          | Description                                |
|--------|---------------|--------------------------------------------|
| POST   | `/auth/login` | Login – returns a signed JWT (24hr expiry) |

```
```json
// Request
{ "email": "admin@artel.com", "password": "artel_2026" }

// Response
{ "access_token": "eyJ...", "token_type": "bearer" }
```
```

### ### Client Data

| Method | Path                         | Description                                         |
|--------|------------------------------|-----------------------------------------------------|
| GET    | `/insights/{client_id}`      | All call rows joined with audio insights            |
| GET    | `/stats/{client_id}`         | Aggregate KPIs, benchmark, hourly breakdown, topics |
| GET    | `/call/{call_id}`            | Single call detail + full transcript with emotions  |
| GET    | `/audio/{call_id}`           | Stream the `.wav` audio file                        |
| GET    | `/feed/{client_id}`          | 30–50 most recent calls (live feed, poll every 5s)  |
| GET    | `/search?q=<term>`           | Full-text transcript search, highlighted snippets   |
| GET    | `/client-alerts/{client_id}` | Call-level alert flags for Home page                |
| GET    | `/emotions/{client_id}`      | Emotion distribution + weekly trend                 |
| GET    | `/topics/{client_id}`        | Topic breakdown with percentages                    |

### ### RAG

| Method | Path                 | Description                                                     |
|--------|----------------------|-----------------------------------------------------------------|
| POST   | `/query`             | Natural language query – `{ question, client_id, history: [] }` |
| POST   | `/embed/{client_id}` | Build / refresh vector store                                    |

### ### Agent Analysis

| Method | Path                         | Description                                    |
|--------|------------------------------|------------------------------------------------|
| POST   | `/agent/analyze/{client_id}` | ReAct agent analysis – `{ query, agent_name }` |
| GET    | `/agent/tools`               | List available tool names and descriptions     |

### ### Admin (heya\_admin only)

| Method | Path | Description |
|--------|------|-------------|
| ---    | ---  | ---         |

| Method | Path                         | Description                                           |
|--------|------------------------------|-------------------------------------------------------|
| GET    | /admin/users                 | List all platform users                               |
| POST   | /admin/users                 | Create a user                                         |
| DELETE | /admin/users/{user_id}       | Delete a user                                         |
| GET    | /admin/clients/overview      | All clients with KPIs, silence ratio, queue depth     |
| GET    | /admin/feed                  | Recent calls across ALL clients (limit=1-100)         |
| GET    | /admin/calls                 | All calls across all clients with full filter support |
| GET    | /admin/agents?client_id=<id> | Agents for a specific client                          |

#### #### /admin/calls` Filter Parameters

| Param      | Type    | Description                         |
|------------|---------|-------------------------------------|
| client_id  | string  | Filter by client                    |
| agent_id   | string  | Filter by agent                     |
| date_from  | string  | ISO date lower bound                |
| date_to    | string  | ISO date upper bound                |
| direction  | string  | inbound or outbound                 |
| flow       | string  | smooth, moderate, or poor           |
| trajectory | string  | improving, stable, or deteriorating |
| sentiment  | string  | Positive, Negative, or Neutral      |
| topic      | string  | keyword topic label                 |
| limit      | integer | default 500                         |

#### ### Export

| Method | Path                       | Description                                      |
|--------|----------------------------|--------------------------------------------------|
| GET    | /export/csv/{client_id}    | Download all calls + insights as CSV (UTF-8 BOM) |
| GET    | /export/report/{client_id} | Printable HTML summary report                    |

#### ### Health

| Method | Path          | Description                        |
|--------|---------------|------------------------------------|
| GET    | /             | API status, version                |
| GET    | /health/db    | PostgreSQL connection check        |
| GET    | /health/queue | Processing queue status per client |

---

#### ## Frontend Pages

##### ### State Architecture

Pages never call the API directly. All data flows through shared mechanisms:

- `useClientData` hook` – fetches `/stats`` and `/insights`` once; returns `{ stats, insights, loading, lastUpdated, refresh }`
- `FilterContext`` – global filter state: date range, flow, direction, topic, `agent`` – persists across page navigation
- `ThemeContext`` – dark/light theme toggle; sets `data-theme`` on `<html>`; persists to `localStorage`` as `heya_theme``

##### ### Client Pages

| Page                | Route                          | What It Shows                                                    |
|---------------------|--------------------------------|------------------------------------------------------------------|
| <code>Home`</code>  | <code>/dashboard/home`</code>  | KPI story cards, 5 charts, call alerts panel, recent calls table |
| <code>Calls`</code> | <code>/dashboard/calls`</code> | Filter bar (incl. agent dropdown), call table with agent         |

column, Analytics/Live toggle, call detail drawer |  
 | **\*\*Audio Insights\*\*** | ``/dashboard/audio-insights`` | 9 analytics sections with charts |  
 | **\*\*Trends\*\*** | ``/dashboard/trends`` | Time-series charts over selectable date ranges |  
 | **\*\*Ask Your Data\*\*** | ``/dashboard/ask`` | RAG multi-turn chat + agent-scoped ReAct analysis section |  
 | **\*\*Feed\*\*** | ``/dashboard/feed`` | Real-time call stream, 5s auto-poll |  
 | **\*\*Alerts\*\*** | ``/dashboard/alerts`` | Alert log and configuration |  
 | **\*\*Search\*\*** | ``/dashboard/search`` | Full-text search with snippet highlights and call detail drawer |

#### #### Calls Page – Agent Filter

The agent dropdown is dynamically built from the ``insights`` data – only agents that appear in the current client's calls are shown. Selecting an agent filters the call table. The Agent column shows ``persona_name`` as an indigo badge.

#### #### Ask Your Data – Two Sections

1. **\*\*RAG Chat\*\*** – general question pills + multi-turn conversation via ``/query``
2. **\*\*Agent Analysis\*\*** – client-scoped agent cards + preset analysis pills → ReAct via ``/agent/analyze/{clientId}``. Agent answers show confidence, tools used, step count, latency, and a collapsible reasoning chain.

#### ### Admin App (4 Tabs)

Accessible only to ``role = heya_admin`` at ``/admin``.

| Tab                     | What It Shows                                                               |
|-------------------------|-----------------------------------------------------------------------------|
| <b>**Overview**</b>     | Client matrix – all KPIs; click any row to open drill-down panel            |
| <b>**Feed**</b>         | Cross-client call feed – 18 columns, 9 filters, CSV export, 5s live refresh |
| <b>**Intelligence**</b> | RAG chat with "Platform Overview (All Clients)" dropdown option             |
| <b>**Users**</b>        | Create / delete platform users, assign to clients                           |

**\*\*Drill-down panel\*\*** (click any client row): 7 analytics bar metrics, call table with Trajectory and Emotion columns, portfolio-level recommendations, cross-client insight callout.

**\*\*Admin Feed – 9 Filters:\*\*** Client → Agent (auto-populates on client select, resets on change) → Date From/To → Direction → Flow → Trajectory → Sentiment → Topic → × Clear

**\*\*Admin Feed – 18 Columns:\*\*** status, client, call ID, agent, direction, datetime, duration, quality grade + score, engagement mini-bar, silence %, interruptions, flow chip, trajectory, sentiment, emotion, topic badge, outcome

---

#### ## Security Model

##### ### Authentication

- All endpoints (except ``/auth/login``) require a ``Bearer`` JWT
- HS256-signed using ``JWT_SECRET_KEY`` from ``.env``
- Token payload: ``{ sub: user_id, client_id, role, exp }``
- 24-hour expiry
- Passwords bcrypt-hashed directly – passlib is not used (breaks with bcrypt 4.x)

##### ### Multi-Tenant Isolation (Three Layers)

| Layer | Mechanism | Where |
|-------|-----------|-------|
| ---   | ---       | ---   |

```
| 1. JWT Claims | `client_id` embedded in signed token; cannot be forged | `auth.py` |
| 2. App check | `_require_client_access(user, client_id)` called manually at each endpoint |
`main.py` |
| 3. PostgreSQL RLS | `heya_app` role has RLS policies; row access blocked at DB level |
`migrate_rls.py` |
```

### ### Roles

```
Role	Access
`heya_admin`	All clients, admin panel, user management, cross-client feed and RAG
`client`	Only their own `client_id` – enforced in JWT, application check, and RLS
```

### ### Agent Analysis Isolation

The agent system prompt in `agent.py` is dynamically built from the database.  
`\_client\_agent\_options(client\_id)` queries which agents belong to the requesting client and injects only those names. A MVAA user cannot see or query Sasha's name.

---

## ## Quality Scoring

Each processed call receives a quality score (0–100) and letter grade (A–F) from `quality.py`.

```
Signal	Weight	Details
Engagement score	35%	Acoustic composite from pipeline
Conversation flow	25%	smooth = 100, moderate = 60, poor = 20
Call outcome	25%	`call_successful = true` → 100, false → 0, null → 50
Customer emotion	15%	happy=100, surprised=75, neutral=60, fearful=35, sad=30,
disgusted=20, angry=15 |
```

**\*\*Grade thresholds:\*\*** A ≥ 80, B ≥ 65, C ≥ 50, D ≥ 35, F < 35

**\*\*Engagement benchmark:\*\*** Voice AI industry baseline fixed at **\*\*62.0\*\***. Clients see their delta on the Home page and in exported reports.

---

## ## Alert System

### ### Call-Level Alerts

Generated by `recommendations.py` → `generate_client_alerts()`, shown on the client Home page.

```
Alert Type	Trigger
Early Disconnect	`duration_ms < 20,000`
Very Poor Engagement	`engagement_score < 25`
Abusive / Escalatory Language	Transcript contains flagged keywords: profanity, "sue",
"refund", "cancel", "manager"	
Extreme Silence	`silence_ratio > 0.65`
High Interruptions	`interruption_count >= 8`
```

### ### Portfolio Recommendations

Generated by `generate\_recommendations()`, shown in admin drill-down panel.

```
| Benchmark | Threshold |
```

```
|---|---|
| Healthy engagement | 65.0 / 100 |
| Critical engagement | 45.0 / 100 |
| Target success rate | 80% |
| Elevated silence ratio | 15% |
| Critical silence ratio | 30% |
| High interruptions (avg) | 3.0 per call |
| Poor flow (warning) | 40% of calls |
| Poor flow (critical) | 65% of calls |
| Negative sentiment (high) | 25% of calls |
```

---

## ## Export & Reports

### ### CSV Export

`GET /export/csv/{client\_id}` – streams a `.csv` with one row per call. Includes all call metadata, audio insight metrics, quality score, and grade. UTF-8 BOM encoded so it opens correctly in Excel.

### ### HTML Report

`GET /export/report/{client\_id}` – printable HTML page with KPI cards, engagement benchmark label, flow breakdown, emotion breakdown.

To save as PDF: open in browser → File → Print → Save as PDF.

---

## ## Known Gotchas

Read before making any changes. These are the non-obvious facts that have caused bugs.

### ### Database

```
Gotcha	Detail
Port is 5435	Not the default 5432. All connection strings must use 5435.
`start_timestamp` has one `t`	Typo from the original Retell export. Aliased to `start_timestamp` in Python. Never rename the column.
RLS uses `heya_app` role	`APP_DATABASE_URL` must connect as `heya_app`. The `postgres` superuser bypasses RLS and should only be used for migrations.
`user_sentiment` is capitalised	DB stores `Positive`, `Negative`, `Neutral` with a capital letter. Always `.toLowerCase()` before comparing in frontend JS/CSS lookups.
```

### ### Backend

```
Gotcha	Detail
`_require_client_access()` is not `Depends()`	Plain function – must be called manually at the top of each endpoint. Intentional for explicit control.
No passlib	`import bcrypt` directly. passlib breaks with bcrypt 4.x hash format.
No LangChain LLM wrappers	`langchain-core 0.3.86` crashes on LLM instantiation in `heya_audio`. Use `openai` SDK with Cerebras `base_url` directly.
`safe_json_float()` for all numerics	Pipeline can produce `NaN`/`Inf`. Always pass numeric fields through this helper before returning in API responses.
`PYTHONIOENCODING=utf-8` on Windows	Emoji in `database.py` causes `UnicodeEncodeError` with cp1252. Set this before starting uvicorn or any pipeline script.
Agent system prompt is dynamic	`agent.py` builds the persona list from the DB via
```



`\_client\_agent\_options(client\_id)`. Never hardcode agent names – that's a cross-tenant leak. |

### ### Frontend

| Gotcha | Detail |  
|---|---|  
| **\*\*No Redux\*\*** | State = `AuthContext` + `useClientData` hook + `FilterContext`. Pages do not call the API directly. |  
| **\*\*`App.jsx` was deleted\*\*** | Entry point is `main.jsx`. Do not recreate `App.jsx`. |  
| **\*\*`App.css` is shared\*\*** | Imported by every page component. CSS changes affect the entire application. |  
| **\*\*`FilterContext` persists across pages\*\*** | Filters set on Calls page are active on Home, and vice versa. `filterAgent` is included in `clearFilters()`. |  
| **\*\*`feedFiltersRef` in Admin Feed\*\*** | The live refresh timer captures filters via a ref (`feedFiltersRef`), not state, so callbacks always see the latest filter values without stale closures. |  
| **\*\*Theme via CSS variables\*\*** | `index.css` defines `:root` (dark) and `[data-theme="light"]` overrides. Do not hardcode colour values in component CSS – use the CSS variable tokens. |

### ### Infrastructure

| Gotcha | Detail |  
|---|---|  
| **\*\*Two conda envs, never merged\*\*** | `heya\_audio` (web) and `heya\_pipeline` (GPU) have conflicting CUDA DLL requirements on Windows. |  
| **\*\*Groq is replaced by Cerebras\*\*** | Any `GROQ\_API\_KEY` references in old code are stale. Use `CEREBRAS\_API\_KEY`. |  
| **\*\*Ollama must be running for RAG\*\*** | Server prints a startup warning if unreachable. All other features work without Ollama. |  
| **\*\*Rebuild RAG store explicitly\*\*** | Run `python rebuild\_rag.py` as an offline step. Never inline vector store rebuilding inside the web server. |

---

### ## Test Suite

**\*\*Location:\*\*** `tests/` – 15 test files  
**\*\*Framework:\*\*** pytest  
**\*\*Config:\*\*** `pytest.ini` at project root

```
```bash
conda activate heya_audio
cd D:\rmit\semester_4\project
pytest tests/ -v
```
```

| Test File                 | Coverage Area                                |
|---------------------------|----------------------------------------------|
| `test_auth.py`            | Login, token generation, invalid credentials |
| `test_health.py`          | Health endpoints                             |
| `test_stats.py`           | Stats endpoint, KPI fields                   |
| `test_insights.py`        | Insights endpoint                            |
| `test_calls.py`           | Call detail, transcript                      |
| `test_admin.py`           | Admin endpoints, role enforcement            |
| `test_alerts.py`          | Alert detection logic                        |
| `test_recommendations.py` | Recommendation thresholds                    |
| `test_quality.py`         | Quality scoring + grade assignment           |
| `test_export.py`          | CSV and HTML report endpoints                |
| `test_search.py`          | Full-text search                             |
| `test_security.py`        | Tenant isolation, 403 enforcement            |

---

```
| `test_database_helpers.py` | DB CRUD helpers, JSON serialisation |
| `conftest.py` | Shared fixtures, test DB setup |
```

```

```

```
Last updated: 2026-05-31
```

```
RMIT COSC2667/COSC2777, Semester 4
```